

Personal AI Operating System

Complete Architecture & Build Guide

Version 1.0

June 9, 2026

Provider-Agnostic Personal AI Platform

Your Use Cases

- **Study** — flashcards, study guides, revision sheets
- **Trading** — research notes, market analysis, insights
- **Excel / Data Analytics** — Q&A on uploaded spreadsheets
- **Research** — storage systems, technical deep-dives
- **General Q&A** — multi-model chat with memory & citations

Table of Contents

1. Complete Architecture & Build Guide
2. 1. Executive Summary
3. 2. System Architecture
4. 3. Functional Requirements
5. 4. Non-Functional Requirements
6. 5. Technology Stack
7. 6. Data Architecture Tradeoffs
8. 7. Database Design
9. 8. Memory Architecture
10. 9. RAG Architecture
11. 10. Model Routing & Fallback
12. 11. Web Search Integration
13. 12. Third-Party Integrations
14. 13. Analytics Architecture
15. 14. API Design
16. 15. Security & Access Control
17. 16. Deployment & Scaling
18. 17. Knowledge & Notes System
19. 18. Folder Structure
20. 19. Build vs Outsource vs Custom
21. 20. MVP Roadmap (8-12 Weeks)
22. 21. Phase-Wise Build Plan
23. 22. Development Milestones
24. 23. Step-by-Step Build Order
25. 24. Architecture Decisions

Personal AI Operating System (PAIOS)

Complete Architecture & Build Guide

Version: 1.0

Date: June 9, 2026

Purpose: Production-grade, provider-agnostic Personal AI OS for study, trading, Excel data analytics, Q&A, research, and storage systems exploration.

Table of Contents

1. [Executive Summary](#1-executive-summary)
 2. [System Architecture](#2-system-architecture)
 3. [Functional Requirements](#3-functional-requirements)
 4. [Non-Functional Requirements](#4-non-functional-requirements)
 5. [Technology Stack](#5-technology-stack)
 6. [Data Architecture Tradeoffs](#6-data-architecture-tradeoffs)
 7. [Database Design](#7-database-design)
 8. [Memory Architecture](#8-memory-architecture)
 9. [RAG Architecture](#9-rag-architecture)
 10. [Model Routing & Fallback](#10-model-routing--fallback)
 11. [Web Search Integration](#11-web-search-integration)
 12. [Third-Party Integrations](#12-third-party-integrations)
 13. [Analytics Architecture](#13-analytics-architecture)
 14. [API Design](#14-api-design)
 15. [Security & Access Control](#15-security--access-control)
 16. [Deployment & Scaling](#16-deployment--scaling)
 17. [Knowledge & Notes System](#17-knowledge--notes-system)
 18. [Folder Structure](#18-folder-structure)
 19. [Build vs Outsource vs Custom](#19-build-vs-outsource-vs-custom)
 20. [MVP Roadmap](#20-mvp-roadmap)
 21. [Phase-Wise Build Plan](#21-phase-wise-build-plan)
 22. [Development Milestones](#22-development-milestones)
 23. [Step-by-Step Build Order](#23-step-by-step-build-order)
 24. [Architecture Decisions](#24-architecture-decisions)
-

1. Executive Summary

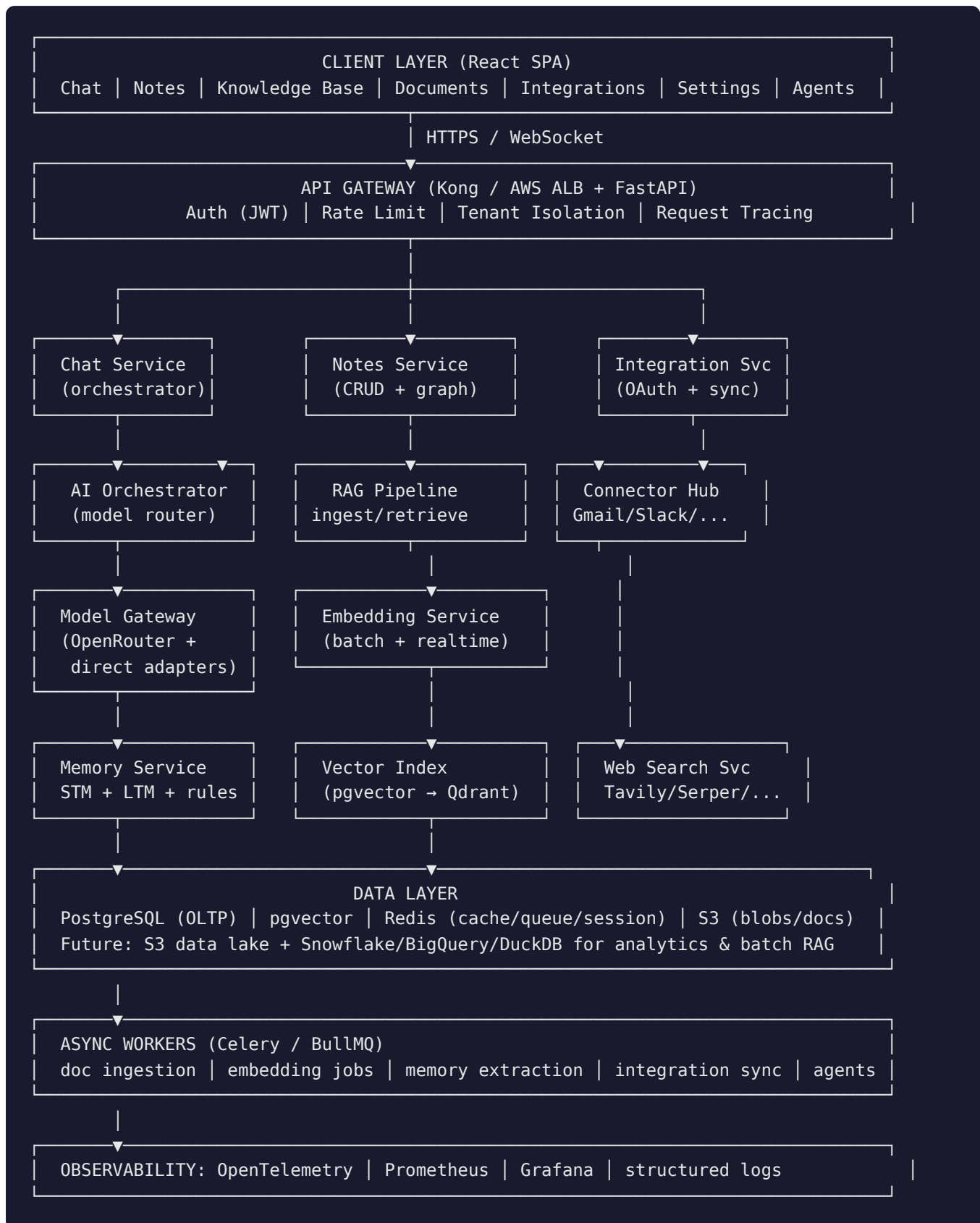
Principle	Decision
Data ownership	All conversations, memory, notes, embeddings, and business data live in **your** PostgreSQL + object storage
Model neutrality	OpenRouter as primary gateway; direct provider adapters as fallback
Separation of concerns	**Memory** = AI personalization; **Notes** = user-owned knowledge artifacts
Build strategy	MVP on PostgreSQL + pgvector + Redis; evolve to lakehouse (S3 + warehouse) for analytics at scale
Integration pattern	OAuth connectors + MCP-style plugin runtime for future extensibility

Use Cases Supported

- **Study:** Flashcards, study guides, revision sheets from conversations
 - **Trading:** Research notes, market analysis, saved insights with semantic search
 - **Excel/Data Analytics:** Document RAG over spreadsheets, Q&A on uploaded data
 - **Research:** Web search + notes + long-term memory for storage systems and technical topics
 - **General Q&A:** Multi-model chat with memory and citations
-

2. System Architecture

2.1 Architecture Diagram



2.2 Chat Request Flow

```
User message
→ API Gateway (auth, rate limit)
→ Chat Service loads conversation + user profile
→ Memory Service: STM window + LTM retrieval (semantic)
→ RAG Service: retrieve from documents + notes (if enabled)
→ Optional: Web Search (if tool triggered or user setting)
→ Optional: Integration context (recent Gmail, Slack threads)
→ Prompt Assembler (system + memory + RAG + tools + user message)
→ Model Router (user-selected model or auto policy)
→ OpenRouter / direct provider (stream response)
→ Stream to client via WebSocket/SSE
→ Post-turn: persist message, extract memories, suggest note saves, log analytics
```

3. Functional Requirements

3.1 Core Platform

ID	Requirement	Priority
FR-01	Users can authenticate and manage profiles	P0
FR-02	Multi-model chat with streaming responses	P0
FR-03	Runtime model selection per conversation or message	P0
FR-04	Conversation history persisted and searchable	P0
FR-05	Short-term context window management	P0
FR-06	Long-term memory extraction and retrieval	P0
FR-07	Document upload and custom RAG	P0
FR-08	Notes system (CRUD, tags, folders, backlinks)	P0
FR-09	Save messages/conversations to notes	P1
FR-10	AI-generated structured notes (summaries, flashcards, etc.)	P1
FR-11	Web search when required	P1
FR-12	Analytics dashboard (usage, cost, latency)	P1
FR-13	OAuth integrations (Gmail, Slack, Notion, Drive, Calendar)	P2
FR-14	AI agents with tool use	P2
FR-15	MCP-style plugin connectors	P3

3.2 Chat Features

- Create/rename/archive/delete conversations
- Branch conversations (fork from message)
- Attach documents to conversations
- Toggle RAG sources (documents, notes, web)
- Select model per chat or override per message
- Regenerate, edit, delete messages
- Export conversation

3.3 Memory Features

- Extract user preferences, facts, goals from conversations
- Semantic recall of older conversations
- Memory confidence scores and decay
- User can view, edit, pin, or delete memories
- **Explicit separation from Notes**

3.4 Notes Features

- Rich text / Markdown notes
- Folders, collections, tags, backlinks (wiki-style `[[links]]`)
- Full-text + semantic search
- Auto-save from chat (message, summary, insight)
- AI transforms: study guide, flashcards, interview Qs, revision sheet
- Version history
- Embeddings for RAG (notes are retrieval sources, not memory)

3.5 Documents & Knowledge Base

- Upload PDF, DOCX, TXT, MD, HTML, XLSX/CSV
 - Folder organization
 - Chunking, embedding, indexing pipeline
 - Citation in chat responses
 - Re-index on update
-

4. Non-Functional Requirements

Category	Requirement	Target
Availability	Core chat API uptime	99.9%
Latency	Time to first token (p95)	< 2s (excl. model)
Latency	RAG retrieval (p95)	< 300ms
Scalability	Concurrent users (MVP)	100; scale to 10K+
Data residency	User data in owned DB/S3	Full control
Security	Encryption at rest + in transit	TLS 1.3, AES-256
Compliance	GDPR delete/export	Right to erasure
Observability	Distributed tracing	100% API requests traced
Cost	Model spend visibility	Per-user, per-model
Portability	Provider swap	< 1 day config change

5. Technology Stack

Layer	Recommendation	Rationale
Frontend	React + TypeScript + Vite	Rich ecosystem, your preference
UI	Tailwind + shadcn/ui	Fast, consistent Personal OS feel
State	TanStack Query + Zustand	Server state + local UI state
Backend	**FastAPI (Python)**	Best for AI/ML pipelines, async, typing
Workers	Celery + Redis	Doc ingestion, embeddings, sync jobs
OLTP DB	PostgreSQL 16 + pgvector	Single store for MVP; you own everything
Cache/Queue	Redis 7	Sessions, rate limits, job queues
Object storage	S3 / MinIO	Documents, exports, raw ingestion
Vector (scale)	pgvector → Qdrant (optional)	Start simple, migrate at ~10M vectors
Model gateway	OpenRouter + LiteLLM	Unified API, fallback, cost tracking
Search	Tavily or Serper + Brave fallback	Production web search APIs
Auth	Clerk or Auth0 (MVP) → custom later	Speed to market
Analytics warehouse	ClickHouse or DuckDB on S3 (Phase 3)	Cost-effective OLAP

6. Data Architecture Tradeoffs

PostgreSQL + pgvector vs S3 + Snowflake-like

Dimension	PostgreSQL + pgvector	S3 + Warehouse (Snowflake-like)
Best for	MVP → mid-scale (< 50M vectors)	Analytics, batch RAG, multi-TB docs
Latency	Low (ms retrieval)	Higher (query spin-up)
Ops complexity	Low	High
Cost at small scale	Low (\$50-200/mo)	High (\$500+/mo minimum)
Transactional integrity	Excellent (ACID)	Eventual (ELT pipelines)
Vector search	Good up to ~10M vectors	Not native; needs external vector DB
Your use case fit	**Start here**	Add for analytics + cold archive

Recommended Hybrid Evolution

Phase 1: PostgreSQL + pgvector + S3 (hot path)

Phase 2: S3 data lake for raw docs + embedding parquet files

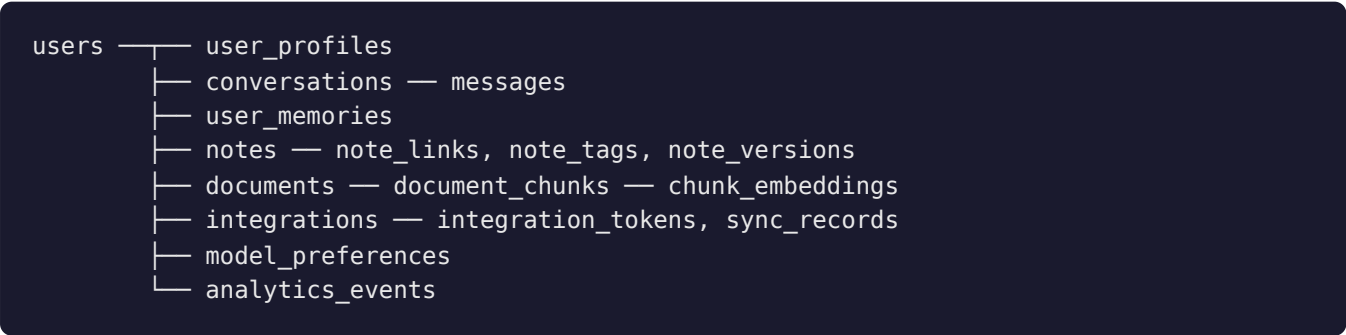
Phase 3: ClickHouse/DuckDB for analytics; Snowflake only if enterprise BI needed

Phase 4: Dedicated vector DB (Qdrant) if pgvector limits hit

Rule: Keep OLTP (users, chats, notes, memories) in PostgreSQL forever. Only offload **analytics** and **cold document archive** to lakehouse.

7. Database Design

7.1 Entity Relationship



7.2 Core Tables

Users & Profiles

```
CREATE TABLE users (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email       TEXT UNIQUE NOT NULL,  
  password_hash TEXT,  
  created_at  TIMESTAMPTZ DEFAULT now(),  
  updated_at  TIMESTAMPTZ DEFAULT now()  
);  
  
CREATE TABLE user_profiles (  
  user_id      UUID PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,  
  display_name TEXT,  
  timezone     TEXT DEFAULT 'UTC',  
  locale       TEXT DEFAULT 'en',  
  preferences  JSONB DEFAULT '{}',  
  created_at   TIMESTAMPTZ DEFAULT now(),  
  updated_at   TIMESTAMPTZ DEFAULT now()  
);
```

Conversations & Messages

```
CREATE TABLE conversations (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  title       TEXT,  
  model_id    TEXT,  
  settings    JSONB DEFAULT '{}',  
  is_archived BOOLEAN DEFAULT false,  
  created_at  TIMESTAMPTZ DEFAULT now(),  
  updated_at  TIMESTAMPTZ DEFAULT now()  
);  
  
CREATE TABLE messages (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  conversation_id UUID NOT NULL REFERENCES conversations(id) ON DELETE CASCADE,  
  role        TEXT NOT NULL CHECK (role IN ('user','assistant','system','tool')),  
  content     TEXT NOT NULL,  
  content_json JSONB,  
  model_id    TEXT,  
  token_count INT,  
  metadata    JSONB DEFAULT '{}',  
  parent_id   UUID REFERENCES messages(id),  
  created_at  TIMESTAMPTZ DEFAULT now()  
);
```

User Memories (AI personalization — NOT notes)

```
CREATE TABLE user_memories (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  memory_type TEXT NOT NULL,  
  content     TEXT NOT NULL,  
  source_type TEXT,  
  source_id   UUID,  
  confidence  REAL DEFAULT 0.8 CHECK (confidence BETWEEN 0 AND 1),  
  relevance_score REAL DEFAULT 1.0,  
  is_pinned   BOOLEAN DEFAULT false,  
  expires_at  TIMESTAMPTZ,  
  embedding   vector(1536),  
  created_at  TIMESTAMPTZ DEFAULT now(),  
  updated_at  TIMESTAMPTZ DEFAULT now(),  
  deleted_at  TIMESTAMPTZ  
);
```

Notes (user-owned knowledge artifacts)

```
CREATE TABLE notes (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  title       TEXT NOT NULL,  
  content     TEXT,  
  content_plain TEXT,  
  folder_id   UUID REFERENCES note_folders(id),  
  note_type   TEXT DEFAULT 'note',  
  source_type TEXT,  
  source_id   UUID,  
  is_pinned   BOOLEAN DEFAULT false,  
  metadata    JSONB DEFAULT '{}',  
  created_at  TIMESTAMPTZ DEFAULT now(),  
  updated_at  TIMESTAMPTZ DEFAULT now(),  
  deleted_at  TIMESTAMPTZ  
);  
  
CREATE TABLE note_folders (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  parent_id   UUID REFERENCES note_folders(id),  
  name        TEXT NOT NULL,  
  sort_order  INT DEFAULT 0  
);  
  
CREATE TABLE note_tags (  
  note_id UUID REFERENCES notes(id) ON DELETE CASCADE,  
  tag     TEXT NOT NULL,  
  PRIMARY KEY (note_id, tag)  
);  
  
CREATE TABLE note_links (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  source_note_id UUID NOT NULL REFERENCES notes(id) ON DELETE CASCADE,  
  target_note_id UUID NOT NULL REFERENCES notes(id) ON DELETE CASCADE,  
  link_type   TEXT DEFAULT 'backlink',  
  UNIQUE(source_note_id, target_note_id)  
);  
  
CREATE TABLE note_chunks (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  note_id     UUID NOT NULL REFERENCES notes(id) ON DELETE CASCADE,  
  chunk_index INT NOT NULL,  
  content     TEXT NOT NULL,  
  metadata    JSONB DEFAULT '{}',  
  embedding   vector(1536)  
);
```

Documents & RAG

```
CREATE TABLE documents (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id     UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  filename    TEXT NOT NULL,  
  mime_type   TEXT,  
  s3_key      TEXT NOT NULL,  
  status      TEXT DEFAULT 'pending',  
  byte_size   BIGINT,  
  metadata    JSONB DEFAULT '{}',  
  created_at  TIMESTAMPTZ DEFAULT now()  
);  
  
CREATE TABLE document_chunks (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  document_id UUID NOT NULL REFERENCES documents(id) ON DELETE CASCADE,  
  chunk_index INT NOT NULL,  
  content     TEXT NOT NULL,  
  page_number INT,  
  metadata    JSONB DEFAULT '{}',  
  embedding   vector(1536)  
);  
  
CREATE TABLE retrieval_logs (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  message_id  UUID REFERENCES messages(id),  
  source_type TEXT NOT NULL,  
  source_id   UUID,  
  chunk_id    UUID,  
  score       REAL,  
  rank        INT,  
  created_at  TIMESTAMPTZ DEFAULT now()  
);
```

Model Routing & Analytics

```
CREATE TABLE model_registry (  
  id TEXT PRIMARY KEY,  
  provider TEXT NOT NULL,  
  display_name TEXT,  
  capabilities JSONB DEFAULT '[]',  
  cost_per_1k_in REAL,  
  cost_per_1k_out REAL,  
  avg_latency_ms INT,  
  is_active BOOLEAN DEFAULT true  
);  
  
CREATE TABLE usage_events (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES users(id),  
  event_type TEXT NOT NULL,  
  model_id TEXT,  
  tokens_in INT,  
  tokens_out INT,  
  cost_usd REAL,  
  latency_ms INT,  
  metadata JSONB DEFAULT '{}',  
  created_at TIMESTAMPTZ DEFAULT now()  
);
```

Integrations

```
CREATE TABLE integrations (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  provider TEXT NOT NULL,  
  status TEXT DEFAULT 'connected',  
  scopes TEXT[],  
  metadata JSONB DEFAULT '{}',  
  created_at TIMESTAMPTZ DEFAULT now(),  
  UNIQUE(user_id, provider)  
);  
  
CREATE TABLE integration_sync_records (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  integration_id UUID REFERENCES integrations(id) ON DELETE CASCADE,  
  external_id TEXT NOT NULL,  
  resource_type TEXT,  
  title TEXT,  
  snippet TEXT,  
  synced_at TIMESTAMPTZ DEFAULT now(),  
  metadata JSONB DEFAULT '{}',  
  UNIQUE(integration_id, external_id)  
);
```

8. Memory Architecture

8.1 Memory Layers

Layer	Storage	TTL	Purpose
STM (Short-Term)	Redis + last N messages in PG	Session / last 20-50 msgs	Immediate chat context
Working Memory	Assembled per request	Ephemeral	Merged STM + retrieved LTM + RAG
LTM (Long-Term)	`user_memories` + embeddings	Durable	Preferences, facts, goals
Episodic	Conversation embeddings (optional)	Durable	Semantic recall of past chats

8.2 Memory vs Notes — Critical Separation

Aspect	Memory	Notes
Owner	Platform (AI context)	User (knowledge artifact)
Purpose	Personalize AI responses	Organize & retain knowledge
User control	View/edit/delete	Full CRUD + organization
RAG role	Injected as system context	Retrieved as citation sources
Auto-created	Yes (extraction pipeline)	Opt-in or explicit save

8.3 Memory Write Rules

WRITE when:

- User explicitly states preference ("I prefer Python", "Call me Alex")
- User states durable fact ("I work at Acme", "My timezone is IST")
- User sets goal ("I'm learning Rust", "Researching new storage systems")
- Assistant infers high-confidence preference (confidence ≥ 0.85) AND user didn't contradict

DO NOT WRITE when:

- Transient context ("I'm tired today")
- Hypotheticals ("If I were...")
- Low confidence (< 0.6)
- Duplicate of existing memory (cosine similarity > 0.92)
- User disabled memory for conversation

8.4 Memory Update Rules

UPDATE when:

- New info contradicts old → supersede (soft-delete old, create new with higher confidence)
- User edits memory in UI → set confidence = 1.0, source = explicit
- Reinforcement: same fact mentioned again → boost confidence (+0.05, cap 1.0)

DECAY:

- Unpinned memories: $\text{relevance_score} *= 0.99$ per 30 days without retrieval
- Retrieved memories: $\text{relevance_score} = \min(1.0, \text{relevance_score} + 0.1)$

8.5 Memory Deletion Rules

DELETE when:

- User explicitly deletes
- GDPR account deletion (cascade)
- Superseded by contradiction
- Expires_at reached (for temporary context memories)
- confidence < 0.3 AND not pinned AND age > 90 days

8.6 Memory Retrieval Flow

1. Embed current user message
2. Vector search user_memories (top 10, min score 0.75)
3. Boost: pinned (+0.2), recent (+0.1), high confidence (+0.1)
4. Optional: episodic conversation search (top 3 conversation summaries)
5. Deduplicate and rank → inject top 5–8 into system prompt (~500 tokens max)

8.7 Post-Turn Memory Extraction (Async Worker)

After assistant response:

1. LLM structured extraction (JSON schema: memories[])
2. Validate against write rules
3. Dedupe against existing embeddings
4. Insert/update user_memories
5. Generate embeddings (batch)

9. RAG Architecture

9.1 Pipeline

Ingest → Parse → Chunk → Embed → Index → Retrieve → Rerank → Assemble → Cite

9.2 Document Ingestion

Step	Implementation
Upload	Presigned S3 URL → `documents` row (status=pending)
Queue	Celery job `process_document`
Parse	Unstructured.io or pdfplumber + python-docx + openpyxl (Excel)
Clean	Strip headers/footers, normalize whitespace
Metadata	page, section, source filename, user_id

9.3 Chunking Strategy

Content Type	Strategy	Size
PDF/DOCX	Recursive character splitter	512 tokens, 64 overlap
Markdown/Notes	Header-aware (H1/H2 boundaries)	400-600 tokens
Excel/CSV	Per-sheet or row-group chunks	300 tokens
Code	AST-aware or function-level	Variable
Email (Gmail)	Per-thread or per-message	300 tokens

9.4 Embedding Generation

- **Model:** `text-embedding-3-small` via OpenRouter
- **Batch:** 100 chunks per API call
- **Store:** `document_chunks.embedding` , `note_chunks.embedding`
- **Versioning:** `embedding_model` column for re-index migrations

9.5 Retrieval

```
Query embedding
→ Parallel search:
  - document_chunks (top 20)
  - note_chunks (top 10) [if notes RAG enabled]
  - integration_sync_records (FTS top 5) [if integrations enabled]
→ Merge candidates (max 30)
→ Metadata filter: user_id, not deleted, status=ready
```

9.6 Reranking

- **MVP:** Cross-encoder via Cohere Rerank API or `bge-reranker-base`

- **Input:** query + top 30 chunks
- **Output:** top 5-8 chunks (score threshold 0.5)

9.7 Prompt Assembly

```
[System]
You are a personal AI assistant. Use the following context when relevant.
Cite sources using [1], [2] notation.

[User Memory]
{memories}

[Retrieved Context]
[1] (document: report.pdf, p.3) {chunk text}
[2] (note: "Rust Study Guide") {chunk text}

[Conversation History]
{last N messages}

[User Message]
{message}
```

9.8 Citation / Traceability

- Every retrieved chunk logged in `retrieval_logs`
- Assistant `metadata.citations[]` on message
- UI renders clickable citations → open document viewer or note

10. Model Routing & Fallback

10.1 Provider Abstraction

Use **LiteLLM** or thin custom adapter with unified interface:

```
async def complete(model_id: str, messages: list, stream: bool = True):
    provider, model = parse_model_id(model_id)
    # openrouter/anthropic/claude-3.5-sonnet
    ...
```

10.2 Routing Policies

Policy	Behavior
manual	User-selected model only; fallback on error
cost	Route to cheapest capable model for task type
latency	Route to lowest p95 latency model
auto	Task classifier picks model (simple → small, complex → large)
user_override	Per-message model override always wins

10.3 Fallback Chain

```
Primary: user-selected model
  ↓ (429, 5xx, timeout 30s)
Fallback 1: user.fallback_chain[0] or platform default
  ↓
Fallback 2: user.fallback_chain[1]
  ↓
Last resort: gpt-4o-mini / claude-haiku (fast, cheap, reliable)
```

10.4 Error Handling

Error	Action
429 Rate limit	Exponential backoff (1s, 2s, 4s) → fallback model
503 Unavailable	Immediate fallback
Context length exceeded	Summarize history → retry with smaller model
Invalid API key	Alert ops; fail with clear user message

10.5 Model Registry (Seed Data)

model_id	Provider	Use Case
openrouter/anthropic/claude-3.5-sonnet	Anthropic	Default quality
openrouter/openai/gpt-4o	OpenAI	Vision, tools
openrouter/google/gemini-2.0-flash	Google	Fast, cheap
openrouter/meta-llama/llama-3.3-70b	Groq via OR	Open, fast
openrouter/deepseek/deepseek-chat	DeepSeek	Cost-efficient reasoning

11. Web Search Integration

11.1 When to Search

Trigger	Mechanism
User enables "Web search" toggle	Always augment query
Model tool call <code>`web_search`</code>	Agent invokes search tool
Heuristic	Query contains "latest", "today", "current", year 2025+
User slash command	<code>`/search {query}`</code>

11.2 Provider Stack

Priority	Provider	Why
Primary	**Tavily API**	Built for AI agents, clean snippets
Fallback	Serper (Google SERP)	Broad coverage
Fallback 2	Brave Search API	Privacy-friendly

11.3 Flow

1. Rewrite query (optional LLM step)
2. Call search API (top 5 results)
3. Fetch page content (readability extraction, max 3 pages)
4. Chunk + inject as `[Web: title](url)` context
5. Log in `retrieval_logs` (`source_type=web`)
6. Cite URLs in response

11.4 Caching

- Redis cache: `search:{hash(query)}` TTL 1 hour

12. Third-Party Integrations

12.1 Per-Integration Plan

Integration	OAuth Scopes	Sync Strategy	Chat Use
Gmail	`gmail.readonly`	Incremental sync via historyId	Summarize unread, search emails
Slack	`channels:history`, `users:read`	Webhook + periodic backfill	Channel summaries, search
Notion	`read_content`	Poll changed pages (cursor)	Import pages to notes/docs
Google Drive	`drive.readonly`	Watch API or poll	Document ingestion to RAG
Google Calendar	`calendar.readonly`	Sync next 30 days events	Calendar Q&A

12.2 MCP-Style Plugin Runtime (Phase 3+)

```
plugins/  
  manifest.json      # name, version, permissions, tools[]  
  connector.py       # implements search, fetch, action handlers  
  
Runtime:  
  - Sandboxed execution  
  - Permission model: read_email, write_calendar, etc.  
  - Tool registry exposed to AI Orchestrator
```

13. Analytics Architecture

13.1 Event Collection

```
Every API call → usage_events (PostgreSQL, hot)  
                → async flush → S3 parquet (cold, daily)  
                → ClickHouse (Phase 3, dashboards)
```

13.2 Key Metrics

Dashboard	Metrics
Usage	Messages/day, active users, conversations
Models	Tokens in/out per model, cost USD
Performance	p50/p95 latency (TTFT, total), error rate
RAG	Retrieval count, avg chunks, rerank scores
Memory	Memories created/updated/deleted, retrieval hit rate
Integrations	Sync success rate, last sync time

14. API Design

14.1 Conventions

- Base: `https://api.yourplatform.com/v1`
- Auth: `Authorization: Bearer <jwt>`
- Pagination: `?cursor=&limit=20`
- Errors: RFC 7807 Problem Details

14.2 Core Endpoints

Auth & Profile

```
POST /auth/register
POST /auth/login
GET /users/me
PATCH /users/me/profile
PUT /users/me/preferences
```

Chat

```
GET /conversations
POST /conversations
GET /conversations/{id}
PATCH /conversations/{id}
DELETE /conversations/{id}
GET /conversations/{id}/messages
POST /conversations/{id}/messages/stream
POST /conversations/{id}/messages/{msg_id}/regenerate
```

Models

```
GET    /models
PUT    /users/me/model-preferences
```

Memory

```
GET    /memories
POST   /memories
PATCH /memories/{id}
DELETE /memories/{id}
POST   /memories/search
```

Notes

```
GET    /notes
POST   /notes
GET    /notes/{id}
PATCH /notes/{id}
DELETE /notes/{id}
POST   /notes/from-message
POST   /notes/ai-transform
GET    /notes/search?q=&semantic=true
GET    /notes/graph
```

Documents & RAG

```
POST   /documents/upload-url
POST   /documents
GET    /documents
DELETE /documents/{id}
POST   /documents/{id}/reindex
```

Integrations

```
GET    /integrations
POST   /integrations/{provider}/connect
DELETE /integrations/{provider}
POST   /integrations/{provider}/sync
```

Analytics

```
GET    /analytics/usage?from=&to=
GET    /analytics/models
GET    /analytics/costs
```

Web Search

```
POST /search
```

15. Security & Access Control

15.1 Authentication

- JWT access tokens (15 min) + refresh tokens (7 days)
- Or Clerk/Auth0 for MVP
- MFA optional (TOTP) in Phase 2

15.2 Authorization

- Row-level security: every query scoped by `user_id`
- PostgreSQL RLS policies on all user tables
- Integration tokens encrypted at application layer

15.3 Data Protection

Data	Protection
At rest	PostgreSQL TDE or disk encryption; S3 SSE-S3
In transit	TLS 1.3 everywhere
Secrets	AWS Secrets Manager / Vault
PII	Minimize storage; support export/delete

15.4 API Security

- Rate limiting: 60 req/min user, 10 chat/min
- Input sanitization (XSS in notes — DOMPurify)
- CORS whitelist
- Prompt injection mitigation: separate system/user boundaries

16. Deployment & Scaling

16.1 MVP Infrastructure

```
Vercel / Cloudflare Pages  (React)
Railway / Fly.io / ECS      (FastAPI + Celery workers)
RDS PostgreSQL 16 + pgvector
ElastiCache Redis
S3 bucket
```

16.2 Production Scaling

Component	Scaling
API	Horizontal pods + HPA
Workers	Queue-depth based scaling
PostgreSQL	Read replicas; PgBouncer
Redis	Cluster mode
Vector search	Migrate to Qdrant if > 10M vectors

16.3 Environments

```
dev → staging → production (separate DB, S3, API keys)
```

17. Knowledge & Notes System

17.1 Note Lifecycle

```
CREATE (manual | from_chat | from_doc | from_integration | ai_generated)
  → DRAFT (auto-save every 2s)
  → PUBLISHED (user saves)
  → INDEXING (async chunk + embed)
  → INDEXED (available for RAG + search)
  → ARCHIVED / DELETED (soft delete, remove from RAG index)
```

17.2 AI Transform Types

Transform	Output note_type	Structure
Summary	summary	Bullet points
Study guide	study_guide	Sections + key concepts
Flashcards	flashcard	JSON {q, a}[] in metadata
Interview Qs	interview_questions	Numbered list
Revision sheet	revision_sheet	Compact facts table
Knowledge article	article	Full structured doc

17.3 UX Flows

Flow A: Save message to note

1. User clicks "Save to Notes" on message
2. Modal: choose folder, add tags, edit title
3. Note created with source_type=chat
4. Toast: "Saved. Indexing for search..."

Flow B: Conversation → Study guide

1. User clicks "Create Study Guide" on conversation
2. Backend runs transform job (selected model)
3. New note appears in folder "AI Generated"

Flow C: Semantic search across notes

1. User types in Notes search bar
2. Toggle: Full-text / Semantic
3. Results ranked with highlights + backlinks preview

18. Folder Structure

```
paios/
├── apps/
│   ├── web/                                # React frontend
│   │   └── src/features/
│   │       ├── chat/
│   │       ├── notes/
│   │       ├── documents/
│   │       ├── memory/
│   │       ├── integrations/
│   │       ├── analytics/
│   │       └── settings/
│   └── api/                                # FastAPI backend
│       ├── app/
│       │   ├── api/v1/
│       │   ├── core/
│       │   ├── services/
│       │   │   ├── orchestrator.py
│       │   │   ├── model_router.py
│       │   │   ├── memory_service.py
│       │   │   ├── rag_service.py
│       │   │   ├── notes_service.py
│       │   │   └── web_search_service.py
│       │   ├── providers/
│       │   ├── integrations/
│       │   └── workers/
│       ├── migrations/
│       └── tests/
├── packages/shared-types/
├── infra/docker/
├── docs/
└── docker-compose.yml
```

19. Build vs Outsource vs Custom

Component	Build	Outsource	Why
Chat UI	✓		Core differentiation
Notes/KB UI	✓		Personal OS experience
Orchestration pipeline	✓		Your IP; provider-agnostic
Memory logic	✓		Core differentiation
RAG pipeline	✓		Data ownership
Model inference		✓ OpenRouter	Commodity
Embeddings API		✓ OpenRouter	Or self-host later
Web search	Thin wrapper	✓ Tavily	Not worth building
Auth (MVP)		✓ Clerk	Speed
Object storage		✓ S3/MinIO	Commodity
Vector DB (MVP)	✓ pgvector		Owned data
Document parsing	Thin wrapper	✓ Unstructured.io	Complex formats

Always custom-owned: conversations, messages, memories, notes, embeddings metadata, user profiles, retrieval logs.

20. MVP Roadmap (8-12 Weeks)

Week 1-2: Foundation

- Monorepo setup, Docker Compose (Postgres+pgvector, Redis, MinIO)
- FastAPI skeleton, auth
- React app shell: layout, routing, auth
- DB migrations: users, conversations, messages

Week 3-4: Chat + Models

- OpenRouter integration via LiteLLM
- Model registry + user model selection
- Streaming chat (SSE)
- Conversation CRUD
- Basic usage logging

Week 5-6: Memory

- STM: last N messages
- Memory extraction worker
- user_memories CRUD UI
- Semantic memory retrieval in chat

Week 7-8: RAG + Documents

- S3 upload flow
- Document parsing + chunking worker
- Embedding pipeline
- RAG retrieval + citations in chat
- Document management UI

Week 9-10: Notes

- Notes CRUD, folders, tags
- Save message to note
- Note chunking + embedding
- Full-text + semantic search
- AI transform: summary + study guide

Week 11-12: Polish + Analytics

- Model fallback chain
- Usage/cost dashboard
- Web search integration
- Error handling, rate limits
- Deploy to staging

21. Phase-Wise Build Plan

Phase 1: Personal AI Core (Months 1-3) — MVP

Phase 2: Knowledge OS (Months 4-5)

- Backlinks + graph view
- Note version history
- All AI transforms (flashcards, interview Qs)
- Conversation branching
- Reranking (Cohere)

- Cost-aware routing policies

Phase 3: Connected OS (Months 6-8)

- Gmail + Google Drive + Calendar integrations
- Slack integration
- Notion import
- Integration data in RAG
- Web search heuristics + caching

Phase 4: Agent Platform (Months 9-11)

- Tool registry (search, notes, email, calendar)
- Agent definitions + runs
- MCP server exposing platform tools
- Plugin manifest system
- Multi-step agent workflows

Phase 5: Scale & Enterprise (Months 12+)

- ClickHouse analytics
 - S3 data lake + batch re-embedding
 - Qdrant migration if needed
 - Team workspaces (multi-tenant)
 - SSO/SAML
 - Audit logs + compliance exports
-

22. Development Milestones

Milestone	Success Criteria
M1: Hello Chat	User can log in, send message, get streamed GPT/Claude response
M2: Model Swap	User switches model mid-conversation, no data loss
M3: Memory Works	AI remembers name/preference across new conversations
M4: RAG Works	Upload PDF/Excel, ask question, get cited answer
M5: Notes Live	Save chat to note, search finds it semantically
M6: Resilient	Provider outage triggers fallback automatically
M7: Connected	Gmail emails appear in chat context
M8: Agent Ready	Agent completes multi-step task with tools

23. Step-by-Step Build Order

1. Infrastructure (Docker, PG, Redis, S3)
2. Auth + user profiles
3. Conversations + messages API
4. Model gateway (OpenRouter) + streaming
5. React chat UI
6. Model selector + registry
7. Usage event logging
8. Memory tables + extraction worker
9. Memory injection in orchestrator
10. Memory management UI
11. Document upload + S3
12. Document processing worker
13. Embeddings service
14. RAG retrieval + prompt assembly
15. Citations in UI
16. Notes schema + CRUD API
17. Notes editor UI (Markdown)
18. Note indexing worker
19. Save-to-note from chat
20. Notes search (FTS + semantic)
21. AI note transforms
22. Web search service
23. Model fallback router
24. Analytics API + dashboard
25. Gmail OAuth + sync
26. Slack, Notion, Drive, Calendar
27. Agent framework
28. MCP plugin runtime

24. Architecture Decisions

1. **FastAPI over NestJS** — AI ecosystem fit
 2. **PostgreSQL + pgvector for MVP** — simplicity, ownership
 3. **OpenRouter as primary gateway** — multi-provider without lock-in
 4. **Memory ≠ Notes** — separate tables, separate retrieval paths
 5. **Async workers for all heavy lifting** — embeddings, ingestion, extraction
 6. **Citations via retrieval_logs** — full traceability
 7. **LiteLLM for provider abstraction** — battle-tested fallback
-

End of Document